Code Explanation

Code Explanation: AWS Glue PySpark Customer Order Pipeline

Overview

This AWS Glue job implements a complete data pipeline that ingests customer and order data from S3, applies data quality rules, implements Slowly Changing Dimension (SCD) Type 2 tracking using Apache Hudi, and generates aggregated analytics.---

Step-by-Step Breakdown

Step 1: Configuration Management

Function: `get_job_parameters()`- Defines all pipeline configuration as a dictionary- Specifies S3 paths for source data (customers, orders) and target locations (curated, analytics)- Sets file formats (CSV for input, Parquet for output)- Configures Glue Data Catalog database and table names- Defines Hudi-specific settings for SCD Type 2 implementation- Enables/disables data quality features (deduplication, null removal)**Key Parameters:**- Source paths use placeholder bucket `s3://adif-sdlc/`- Hudi uses `CustId` as record key and `OpTs` (operation timestamp) as precombine key- COPY_ON_WRITE table type for read-optimized queries---

Step 2: Spark Context Initialization

Function: `initialize_spark_contexts()`- Attempts to retrieve job name from Glue environment arguments- Falls back to local mode if not running in Glue (for testing)- Creates SparkContext, GlueContext, and SparkSession- Initializes Glue Job object for tracking and commits- Returns all contexts for use throughout the pipeline---

Step 3: Data Ingestion

Class: `DataReader`**Customer Data Reading:**- Enforces strict schema: `CustId`, `Name`, `EmailId`, `Region` (all strings, non-nullable)- Reads CSV files with header and UTF-8 encoding- Schema enforcement prevents type mismatches**Order Data Reading:**- Reads CSV with initial string schema for all fields- Performs explicit type casting: - `PricePerUnit` → Decimal(10,2) for precise currency calculations - `Qty` → Integer - `Date` → Date type using `to_date()` function- Type casting after read allows handling of malformed data gracefully---

Step 4: Data Quality Processing

Class: `DataCleaner`**Null Removal:**- Filters out records with actual NULL values- Also removes string representations: 'Null', 'null', 'NULL', 'None'- Applies to all columns by default or specified subset- Uses case-insensitive matching for string columns**Deduplication:**- Uses Spark's `dropDuplicates()` method- Can deduplicate on all columns or specified subset- Keeps first occurrence of duplicate records**Combined Cleaning:**- `clean_dataframe()` applies both rules based on configuration flags- Allows selective enabling/disabling of quality checks---

Step 5: SCD Type 2 Implementation

Class: `SCDProcessor`**Adding SCD Columns:**- `IsActive` (Boolean): Marks current version of record- `StartDate` (Timestamp): When this version became active- `EndDate` (Timestamp): When this version was superseded (NULL for active records)- `OpTs` (Timestamp): Operation timestamp for Hudi precombine logic**SCD Type 2 Logic:**1. **First Load:** All records marked as active with current timestamp2. **Incremental Loads:** - Joins new data with existing active records on `CustId` - Compares `Name`, `EmailId`, `Region` to detect changes - **Changed records:** Old version marked inactive with `EndDate`, new version inserted as active - **Unchanged records:** Remain active without modification - **Inactive records:** Preserved in history3. **Result:** Complete history with one active version per customer---

Step 6: Hudi Integration

In: `main()` function**Hudi Configuration:**- Table name: `customer_scd_hudi`- Record key: `CustId` (unique identifier)- Precombine key: `OpTs` (resolves conflicts by timestamp)- Table type: COPY_ON_WRITE (optimized for read-heavy workloads)- Operation: UPSERT (insert new, update existing)**Hive Sync:**- Automatically registers table in Glue Data Catalog- Uses `NonPartitionedExtractor` (no partitioning)- Enables querying via Athena/Spark SQL**Write Process:**- Hudi handles deduplication using record key- Maintains file-level indexes for fast upserts- Stores data in columnar Parquet format with transaction logs---

Step 7: Aggregation Calculation

Class: `AggregationEngine`**Customer Spend Calculation:**1. Creates `TotalSpend` column: `PricePerUnit × Qty`2. Groups by `CustId` and `Date`3. Sums `TotalSpend` for each customer-date combination4. Result: Daily spending per customer**Output Schema:**- `CustId` (String)- `Date` (Date)- `TotalSpend` (Decimal)---

Step 8: Data Catalog Registration

Class: `CatalogManager`**Registration Process:**1. Creates database if not exists2. Drops existing table (ensures clean state)3. Writes DataFrame to S3 in specified format4. Registers location as external table in Glue Data Catalog**Registered Tables:**- `customer_cleaned`: Cleaned customer data- `order_cleaned`: Cleaned order data- `customer_scd_hudi`: SCD Type 2 customer history (via Hudi sync)- `customeraggregatespend`: Daily customer spending aggregates---

Step 9: Data Writing

Class: `S3Writer`**Format Dispatch:**- **Parquet:** Columnar format, supports partitioning (used for aggregates partitioned by Date)- **CSV:** Text format with headers (optional output format)- **Hudi:** Transactional data lake format (used for SCD Type 2)**Write Modes:**- `overwrite`: Replaces existing data- `append`: Adds to existing data (Hudi default)**Validation:**- Checks S3 path format before writing- Prevents writes to invalid locations---

Step 10: Main Pipeline Execution

Function: `main()`**Execution Flow:**1. Initialize Spark contexts2. Load configuration parameters3. **Ingest** customer and order CSV data from S34. **Clean** data (remove nulls, deduplicate)5. **Register** cleaned data in Glue Data Catalog6. **Implement** SCD Type 2 with Hudi for customer dimension7. **Calculate** customer aggregate spending by date8. **Write** aggregates to analytics layer (partitioned Parquet)9. **Register** aggregate table in catalog10. **Commit** Glue job (marks successful completion)**Error Handling:**- Try-except block catches all exceptions- Prints full stack trace for debugging- Re-raises exception to mark job as failed- Finally block ensures Spark cleanup (in

local mode)---

Key Design Patterns

1. **Separation of Concerns:** Each class handles one responsibility (reading, cleaning, writing, etc.)2. **Configuration-Driven:** All paths and settings externalized to parameters3. **Schema Enforcement:** Explicit schemas prevent data type issues4. **Idempotency:** Overwrite mode and table drops ensure repeatable runs5. **Traceability:** Print statements track progress through pipeline stages6. **Production-Ready:** Proper error handling, validation, and job commits---

Data Flow Summary

```
CSV Sources (S3) ↓[Ingest with Schema] ↓[Data Quality Cleaning] ↓■■→ [Curated
Layer] → Glue Catalog (customer_cleaned, order_cleaned)■■→ [SCD Type 2 + Hudi]
→ Glue Catalog (customer_scd_hudi)■■→ [Aggregation] → Analytics Layer → Glue
Catalog (customeraggregatespend)
```